

DSA Assignment

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <ctype.h>


#define MAX 100


// Stack structure
typedef struct {
    char items[MAX];
    int top;
} Stack;


// Function to initialize stack
void init(Stack *s) {
    s->top = -1;
}


// Check if stack is empty
int isEmpty(Stack *s) {
    return s->top == -1;
}


// Push element onto stack
void push(Stack *s, char c) {
```

```
if (s->top == MAX - 1) {  
    printf("Stack Overflow\n");  
    exit(1);  
}  
s->items[++(s->top)] = c;  
}
```

// Pop element from stack

```
char pop(Stack *s) {  
    if (isEmpty(s)) {  
        printf("Stack Underflow\n");  
        exit(1);  
    }  
    return s->items[(s->top)--];  
}
```

// Peek top element of stack

```
char peek(Stack *s) {  
    return s->items[s->top];  
}
```

// Function to return precedence of operators

```
int precedence(char op) {  
    if (op == '^') return 3;  
    if (op == '*' || op == '/') return 2;  
    if (op == '+' || op == '-') return 1;
```

```

    return 0;
}

// Function to check if character is operator
int isOperator(char c) {
    return (c == '+' || c == '-' || c == '*' || c == '/' || c == '^');
}

// Function to convert infix to postfix
void infixToPostfix(char infix[], char postfix[]) {
    Stack s;
    init(&s);
    int i, j = 0;
    char c;

    for (i = 0; infix[i] != '\0'; i++) {
        c = infix[i];

        // If operand, add to output
        if (isalnum(c)) {
            postfix[j++] = c;
        }

        // If '(', push to stack
        else if (c == '(') {
            push(&s, c);
        }
    }
}

```

```

// If ')', pop until '(' is found
else if (c == ')') {
    while (!isEmpty(&s) && peek(&s) != '(')
        postfix[j++] = pop(&s);
    pop(&s); // remove '('
}

// If operator
else if (isOperator(c)) {
    while (!isEmpty(&s) && precedence(peek(&s)) >= precedence(c))
        postfix[j++] = pop(&s);
    push(&s, c);
}
}

// Pop remaining operators
while (!isEmpty(&s))
    postfix[j++] = pop(&s);

postfix[j] = '\0';
}

int main() {
    char infix[MAX], postfix[MAX];

    printf("Enter an infix expression: ");
    gets(infix);

```

```
infixToPostfix(infix, postfix);
```

```
printf("Postfix expression: %s\n", postfix);
```

```
return 0;
```

```
}
```